

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Projekt - Riadenie mobilných robotov

Dokumentácia

Študijný program:	Robotika a kybernetika
Predmet:	I-RMR – Riadenie mobilných robotov
Prednášajúci:	prof. Ing. František Duchoň, PhD.
Cvičiaci:	Ing. Martin Dekan, PhD.

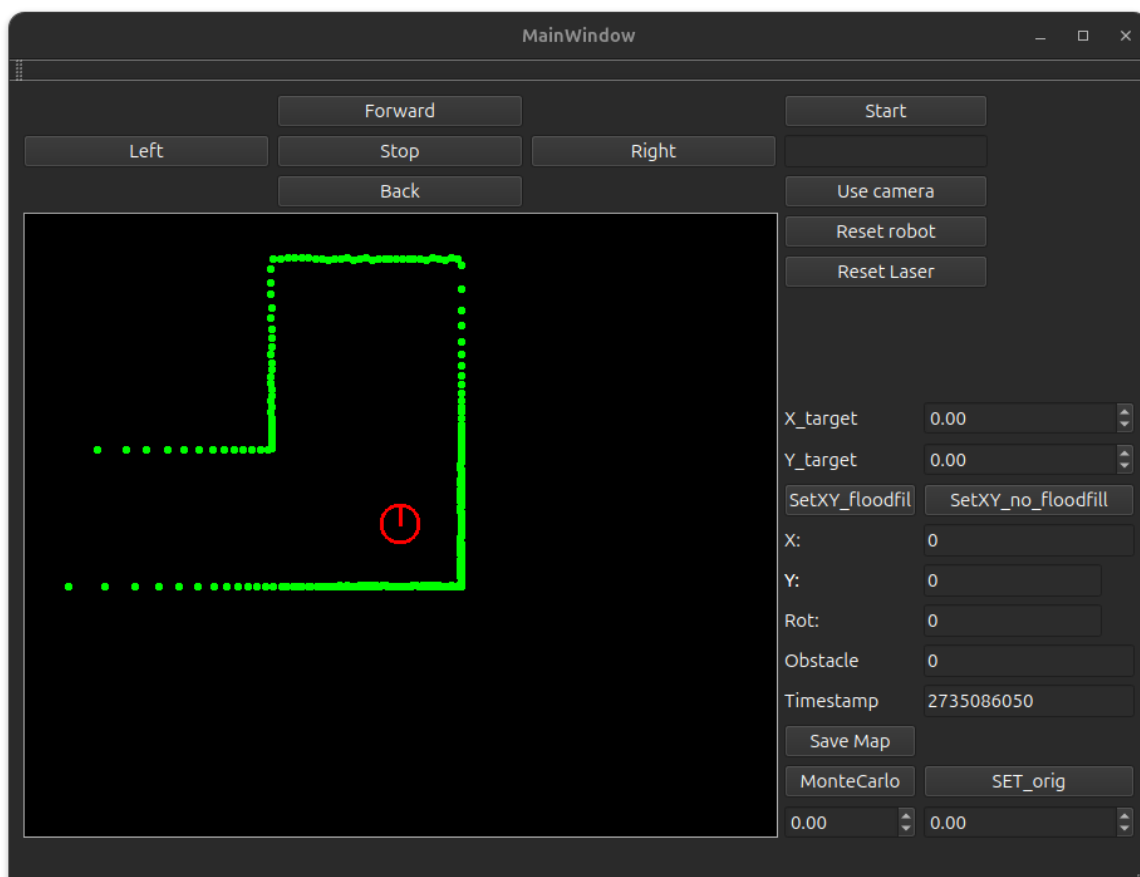
Obsah

1	Užívateľská príručka	3
2	Fungovanie algoritmov - Úloha 1: Lokalizácia a polohovanie robota v prostredí	5
2.1	Lokalizácia (updateOdometry)	5
2.2	Pohyb na cieľ (updateArcTrajectory pre obstacle_detected == false) .	5
3	Fungovanie algoritmov - Úloha 2: Navigácia a vyhýbanie sa prekážkam	6
3.1	Lidar a prekážky (obstacleDetector a rozdelenie do segmentov)	6
3.2	Hľadanie cesty (candidateDirection a Fitness funkcia)	6
3.3	Vyhýbacie manévry (updateArcTrajectory pre obstacle_detected == true)	7
4	Úloha 3: Mapovanie	8
4.1	Vytváranie mapy - mapping.cpp	8
4.2	Vizualizácia mapy - mapvisualizer.cpp	8
5	Fungovanie algoritmov - Úloha 4: Globálna navigácia po známej mape (Floodfill)	9
5.1	Transformácie mapy a robota	9
5.2	Vytvorenie Waypointov pomocou Breadth-First Search (BFS)	9
5.3	Sledovanie Waypointov s reflexiou neočakávaných prekážok	9
6	Fungovanie algoritmov - Úloha 5: Monte Carlo lokalizácia (MCL)	11
6.1	Inicializácia hypotéz (initParticlesGlobal)	11
6.2	Ohodnocovanie (Fitness a Transform)	11
6.3	Koleso rulety číry preber (Roulette a Cutoff)	11
6.4	Prostredie posunov (Noise a Move)	12

1 Uživatelská příručka

Program sa spúšťa prostredníctvom vývojového prostredia Qt a beží ako natívna aplikácia. K svojmu plnohodnotnému fungovaniu vyžaduje nadviazanie spojenia so simulátorom Kobuki (alebo s reálnym robotom), ktorý mu poskytuje potrebné dáta senzoriky a prijíma príkazy pre motory. Všetky detaily týkajúce sa prerekvizít, prvotného nastavenia projektových súborov a samotnej inštalácie sú podrobne popísané v samostatnom inštaláčnom manuáli k projektu.

Softvér disponuje grafickým užívateľským rozhraním určeným na zadávanie cieľov a spúšťanie vnútorných procesov robota.



Obr. 1: Ukážka užívateľského rozhrania

Na obrázku 1 je znázornené užívateľské rozhranie programu. V strednej časti je zobrazený robot a body, ktoré sú detekované pomocou lidar. V hornej časti sa nachádzajú pôvodné tlačidlá pre ovládanie robota (**Forward**, **Backward**, **Left**, **Right**, **Stop**) a v pravom hornom rohu sú tlačidlá **Start**, **Use camera**, **Reset robot**, **Reset Laser**. Tlačidlom **Start** sa spúšťa hlavný cyklus programu a pripojenie sa k IP adrese robota. Textové polia **X_target** a **Y_target** slúžia na zadávanie súradníc cieľa, ktorý sa načíta

do poľa súradníc po stlačení tlačidla `SetXY_no_floodfill`, ktoré zapíše body priamo do poľa, alebo `SetXY_floodfill`, ktoré zavolá floodfill algoritmus. Textové polia `X`, `Y` a `Rot` zobrazujú aktuálnu pozíciu a orientáciu robota. `Obstacle` signalizuje, či sa robot nachádza v móde vyhýbania sa prekážkám. `Timestamp` zobrazuje aktuálnu časovú vzorku. Tlačidlo `Save Map` umožňuje uložiť aktuálne zmapovaný priestor do súboru. Tlačidlo `Monte Carlo` spouští lokalizačný algoritmus Monte Carlo, ktorého výstup sa zobrazuje v konzole. Tlačidlo `SET_orig` nastavuje robotu jeho aktuálnu pozíciu na súradnice `X` a `Y` z textových polí pod ním.

V súčasnosti program plne podporuje ovládanie pomocou nasledujúcich prvkov užívateľského rozhrania:

- **Textové polia pre Zadanie X a Y súradníc (Target):** Dve textové polia určené na vpisovanie koordinát. Vstupom sú údaje v metroch (x a y poloha).
- **Tlačidlo na odoslanie cieľa (Set Target):** Po kliknutí na toto tlačidlo (alebo podobné tlačidlo nastavenia cieľa) sa aktuálne vyplnené koordináty prečítajú a nastaví sa pre internú pamäť robota (funkcia `setTargetXY()`). Robot automaticky vyráta dráhu k cieľu a rozbehne sa určeným smerom.
- **Tlačidlo pre mód lokalizácie (Find Yourself):** Odštartuje beh sekvencie v samostatnom vlákne pre MonteCarlo lokalizáciu robota a dočasne prepne robota do módu domnelej prekážky, počas ktorého robot pomalšie preskúma okolie, až kým nepotvrdí zistenie svojej vlastnej polohy.

2 Fungovanie algoritmov - Úloha 1: Lokalizácia a polohovanie robota v prostredí

Táto sekcia popisuje funkcie pre bezkolízny priamočiary pohyb a sledovanie vlastnej polohy na základe odometrie.

2.1 Lokalizácia (`updateOdometry`)

Ide o základný stavebný kameň lokalizácie, ktorý sa stará o presné počítanie polohy x , y a natočenia z enkodérov a gyroskopu robota. Výpočet využíva diferencie hodnôt medzi jednotlivými volaniami. Enkodéry vrátia delta polohy na ľavom a pravom kolese, z ktorých je vypočítaná priemerná prejdená vzdialenosť (translácia).

2.2 Pohyb na cieľ (`updateArcTrajectory` pre `obstacle_detected == false`)

Bezpečný pohyb po priestore zaručuje trajektória riadená pomocou tejto metódy, pokiaľ lidazr nezaregistroval prekážku. Algoritmus skontroluje vzdialenosť do cieľa prostredníctvom pomocných matematických funkcií:

- `calculateAngleToTarget`: Nájde najkratší uhol medzi aktuálnou rovinou (x_{curr}, y_{curr}) a koncovým cieľom použitím arkus tangens derivátu (`atan2`).
- `normalizeAngle`: Pripočítava alebo odpočítava konštantu 2π , aby uhol zotrval vo formáte $(-\pi, \pi)$.

O pootočenie okolo vlastnej osi za behu do daného smeru sa stará funkcia `piRegulator(error)`. Do regulátora vstupuje uhlová odchýlka (`heading_error`) a na základe K_p (Proportional) a K_i (Integral) konštant sa vracia rýchlosť natáčania s anti-windup podmienkou (ohraničenie akumulovanej chyby v integrátore).

Zároveň kvôli stabilite je volaná funkcia `applySpeedRamp(current, target, max_speed...)`, ktorá implementuje S-krivku (zrýchľovací/spomaľovací profil polynomiálneho radu) pre elimináciu trhania robota a stabilný plynulý nábeh i dobeh k minimu na rýchlosti pohybu.

3 Fungovanie algoritmov - Úloha 2: Navigácia a vyhýbanie sa prekážkam

Ak sa robot po ceste k cieľu stretne s objektom, prechádza plynulo do módu vyhýbania a obchádzania stien.

3.1 Lidar a prekážky (obstacleDetector a rozdelenie do segmentov)

Lidar vracia veľké množstvo bodových meraní v rôznych uhloch. Vzorka merania pokrýva plné okolie robota. Úlohou `obstacleDetector` je tieto body normalizovať a vyhodnotiť na základe hraničných hodnôt vzdialeností (tzv. *threshold_mm* a *hysteresis_mm*). Setuje bool hodnotu `obstacle_detected = true`, čím signalizuje systému že nemôže pretrvávajúť v priamočiariom pohybe k cieľu bez kolízie a mení interný stav pre výber trasy.

Na optimalizáciu je 360 stupňov rozdelených na 80 sektorov (každých 4,5°). Metóda `updateLidarSegments` v spolupráci s `rotateLidarSegmentsBySteps` v `updateOdometry` vykonáva stabilizáciu tohto mapovaného okolia - tzn. fixujú dáta napriek uhlu naklonenia robota použitím hodnôt iterácií delty z gyroskopu a rotujú kandidátske pole, takže index poľa prislúcha v určitom okamihu absolútnemu uhlu voči globálnej rovine, a nie k osi robota.

3.2 Hľadanie cesty (candidateDirection a Fitness funkcia)

Vo chvíli objavenia prekážky musí robot zmeniť svoj cieľový smer. Metóda `candidateDirection` prehľadáva prekážkové segmenty. Identifikuje priechodnosti kandidátskych smerov, na ktoré nasadzuje prispôsobený algoritmus ohodnotenia (Fitness).

Konkrétna implementácia fitness hodnotí vhodnosť daného voľného smeru:

$$Fitness = \mu_1 \cdot finish_distance + \mu_2 \cdot rotate_distance + \mu_3 \cdot change_distance$$

Kde:

- `finish_distance`: Vzdialenosť uhla medzi kandidátom a cieľom, kam chce robot dôjsť.
- `rotate_distance`: Potrebná fyzická výchylka robota k pootočeniu vlastnej úvahovej osi (penalizácia za drastické točky).

- `change_distance`: Vzdialenosť pri zmene ohodnoteného bodu z predošlého výpočtu. Slúži ako histeréza, aby robot stabilne držal aktuálne zvolenú stranu vyhýbania a nezmieňal prudko strany pri identickom fitness ohodnotení pravého či ľavého obchádzania (napr. stĺpa).

Kandidát s najnižšou cenou (Score) sa ukladá.

3.3 Vyhýbacie manévry (`updateArcTrajectory` pre `obstacle_detected == true`)

Podobne ako v prvej úlohe prechádza algoritmus na výpočet obmedzenej rýchlosti. Robot riadi `rotationspeed` opätovným využitím funkcie PI regulácie `piRegulator`, avšak tento raz chybu vyčísluje na základni uhlovej odchýlky napočítanej od víťazného `candidateDirection()`.

4 Úloha 3: Mapovanie

Mapovanie prebieha od spustenia programu a počas celého behu. Lidarové dáta sa na základe aktuálnej polohy robota a jeho orientácie transformujú do mapy. Mapa je reprezentovaná ako pole, kde 0 značí voľný priestor a 255 značí detekovaný objekt alebo stenu.

4.1 Vytváranie mapy - mapping.cpp

4.2 Vizualizácia mapy - mapvisualizer.cpp

5 Fungovanie algoritmov - Úloha 4: Globálna navigácia po známej mape (Floodfill)

Táto časť popisuje spôsob implementácie globálnej navigácie, ktorá dokáže narábať s mapou prekážok a na ktorej sa robot dokáže lokalizovať. Algoritmus floodfill poskytuje robustnejší postup na vyhýbanie sa prekážkam na mape, čím priamo nahrádza lokálnu navigáciu a slepé obchádzanie z úlohy 2 pre známe steny a bloky.

5.1 Transformácie mapy a robota

Základným predpokladom úspešnej dráhy je znalosť vlastnej polohy a pozície na mape. V tejto chvíli je manuálne vložená hodnota z lokálneho súradnicového systému robota (0, 0). Do softvéru vstupujú transformácie, ktoré umožňujú interpolovať a prepisovať reálne pozície robota a polohy jednotlivých pixelov (hodnot v mape charakterov) mapy zobrazených cez mriežku (grid). Týmto transformáciami vieme určiť jednoznačnú pozíciu poľa a vzdialenosti prvkov.

5.2 Vytvorenie Waypointov pomocou Breadth-First Search (BFS)

Celá voľná zóna je reprezentovaná pre algoritmus ako sieť prepojených bodov v gride (t.j. hrany medzi poľami mapy nemajú rôznu dĺžku ale všetky stoja rovnako). Graf hľadá cestu naprieč mapou tak, že sa v iteratívnych prechodoch značí susedné čísla (či už steny alebo voľný priestor, aby viackrát neprehľadalo ten istý pixel) na okolité susedné vrcholy, až dokým nenatrafí na character F (finish).

Výstupom tohto algoritmu sú *waypointy* (určené lámaním uhlov), inak zlomové body na mape. Tie sú generované ako trasa k samotnému koncu a sú zasielané ako sústava postupných malých cieľov `x_target` a `y_target`.

5.3 Sledovanie Waypointov s reflexiou neočakávaných prekážok

Akonáhle dostane štruktúra `robot` a jeho metódy na starosť tieto waypointy, začne ich sledovať použitím lokálneho kontroléru popísaného v Úlohe 1 (presun v móde `obstacle_detected = false`).

Robot nasleduje zadané úseky, no zároveň aktívne stráži okolie cez lidar pre prípad neočakávaných prekážok. Ak sa táto prekážka objaví, robot setne `obstacle_detected = true` a prejde do módu vyhýbania sa, pričom bude pokračovať v sledovaní aktuálneho

waypointu. Po úspešnom obídení prekážky sa vráti do módu priamočiareho pohybu a bude pokračovať v ceste k waypointu, až kým nedosiahne cieľ.

6 Fungovanie algoritmov - Úloha 5: Monte Carlo lokalizácia (MCL)

V prípadoch, keď počiatočná poloha robota na mape nie je jednoznačne určená alebo po strate presnosti odometrie z dlhodobého jazdenia hrozí zablúdenie, systém pristupuje k proaktívnemu hľadaniu vlastnej globálnej polohy za pomoci Monte Carlo lokalizácie (často označovanej aj ako Particle Filter).

Mechanizmus využíva samostatné asynchrónne vlákno (trieda `MonteCarlo`), ktoré simuluje možné hypotézy o polohe robota v podobe mračna "častíc" (particles). Algoritmus zbiera slučku viacerých výpočtových sub-rutín, až keď nepotvrdí zistenie svojej vlastnej polohy (`foundMyself = true`).

6.1 Inicializácia hypotéz (`initParticlesGlobal`)

Na začiatku sa po priestore voľných zón (funkcia `isFreeSpace()`), ktoré robot extrahuje z matice mapy znakov `finalMap.txt`, rozhodí špecifikovaný počet častíc (štandardne v zadání 100). Každý iterácii častice sa vygeneruje náhodná počiatočná pozícia x , y a náhodná rotácia θ s počiatočnou váhou $error = 0$.

6.2 Ohodnocovanie (Fitness a Transform)

V hlavnej slučke MCL systému sa pre každú hypotetickú časticu porovnávajú dáta namerané jej reálnym Lidarom na robote s teoretickým modelom vzdialeností načítaným z textového súboru `distanceMap.txt`. Častice "vrhajú" virtuálne lúče od seba von a prepočítavajú sa na mriežkový grid senzorických polí (prostredníctvom pomocno-geometrickej metódy `transform()`, ktorá zohľadňuje uhlovú posunutú orientáciu θ a normalizované pravidlá rotácie). Výsledky vzdialeností z bunky na stene penalizujú celkovú presnosť (váš `p.error`) danej častice. Ak častica vygeneruje lúč smerujúci von z mapy, obdrží výrazný penalizačný prídelenok (+200). Celková metrika vzdialeností je zároveň uložená v častici do hodnoty `weight`.

6.3 Koleso rulety číry preber (Roulette a Cutoff)

Filtrovací systém ďalej re-alokuje nové generácie vzoriek cez metódu opakovaného prevzorkovania zvaného Kolo rulety (*Resampling wheel, SUS*). Častica, ktorá mala obrovskú chybu (a bola odhalená ako veľmi nepravdepodobná varianta polohy) má malú šancu na sklonovanie. Menší error priamo úmerne prevyšuje pravdepodobnosť uloženia a posunutia hypotézy.

Nasleduje filter `cutoff()`, ktorý brutálne orezáva hypotézy idúce cez múry s obrovským nakumulovaným errorom nad hranicu 10000.0. Tie sú automaticky prepísané úplne nanovo skokom cez randomné znovuspustenie - rovnako ako v inicializačnej `initParticlesGlobal()`.

6.4 Prostredie posunov (Noise a Move)

Pre udržanie lokálnej roznorodosti hypotéz sa pred ďalšou iteráciou nasadzuje `noise()`. Vytvára v častici jemný Gaussiansky šum na pozíciách a uhloch (zabraňuje tomu, aby sa všetky dobré hypotézy zgrupili do priam identického matematického bodu kvôli rulete preberov). V poslednom kroku metóda `move()` upravuje polohu častice posunom o časticovú rotáciu a transláciu prekopírovaným predchádzajúcim pohybom robota (`forwardspeed` na `dt` zlomky, napr. 1/40 sekundy slučiek vlákna).